PIMScope: Augmenting Ramulator 2.0 with Command-Level LPDDR-PIM for Transformer Inference

Ting Lin, Runxi Wang, Jani Babu Shaik, and Xinfei Guo*
Global College, Shanghai Jiao Tong University, Shanghai, China
{ting_lin, wangrunxi, skjanibabu786, xinfei.guo}@sjtu.edu.cn

***Abstract*—Processing-in-memory (PIM) addresses the memory-bandwidth bottleneck of autoregressive transformer decoding by placing compute inside the DRAM banks. On mobile and edge SoCs, LPDDR-PIM is a natural candidate for on-device inference. Yet no open-source command-level LPDDR-PIM backend that simulates full transformer command traces has been released. We present *PIMScope*, an open-source command-level LPDDR-PIM backend built on Ramulator 2.0, a widely-used cycle-accurate DRAM simulator. *PIMScope* adds native PIM opcodes, a shared-PIM-block model central to commercial LPDDR-PIM architecture, and a two-level timing model with a structural energy interface that separates JEDEC-grounded LPDDR costs from PIM-specific coefficients. A three-stage trace-lowering pipeline converts decoder-only transformer specifications into backend-executable LPDDR-PIM command streams. We evaluate PIMScope on 15 decoder-only transformer models from five families. The backend completes all traces with command matching, quantifies a 1.17–$1.80\times$ slowdown for the two-bank shared-PIM-block design over a single-bank design, and measures weight-preload overhead at 18–38% of steady-state cycles in decode versus 2–3% in prefill, pointing that weight placement and residency are critical for performance optimization.**

***Index Terms*—LPDDR-PIM, Processing-in-Memory, Ramulator 2.0, DRAM Simulator, Simulation Infrastructure**

I. Introduction & Motivation

Dynamic Random-Access Memory (DRAM) is the dominant main memory in modern computing systems, but its bandwidth and access latency have not scaled with processor compute [1]. The resulting "memory wall" makes data movement a dominant performance and energy cost for data-intensive workloads [2]. On-device transformer inference exposes this acutely during autoregressive decoding, where tokens are generated one at a time and each step must re-fetch model parameters and the key-value (KV) cache from memory [3]. Prior PIM-based LLM studies characterize this decode phase as severely memory-bound: multi-head attention issues bandwidth-heavy matrix-vector multiplications (GEMV) over request-specific activations, while feed-forward and projection layers behave as compute-intensive matrix-matrix multiplications (GEMM) only at larger batch sizes [4]–[6].

On mobile and edge System-on-Chips (SoCs), this main memory is Low-Power Double Data Rate (LPDDR) DRAM, engineered for the power and thermal envelopes of battery-powered devices. The widely deployed LPDDR5/5X generation raises peak pin bandwidth over LPDDR4, with LPDDR5X reaching 8.533 Gbps, while preserving the low-power interface mobile SoCs depend on [8]–[10]. Even so, the decode bottleneck persists: the LPDDR channel saturates while the on-chip accelerator's arithmetic units sit largely idle. LPDDR is thus both the dominant edge main memory and the natural target for memory-system techniques that relieve the decode bottleneck.

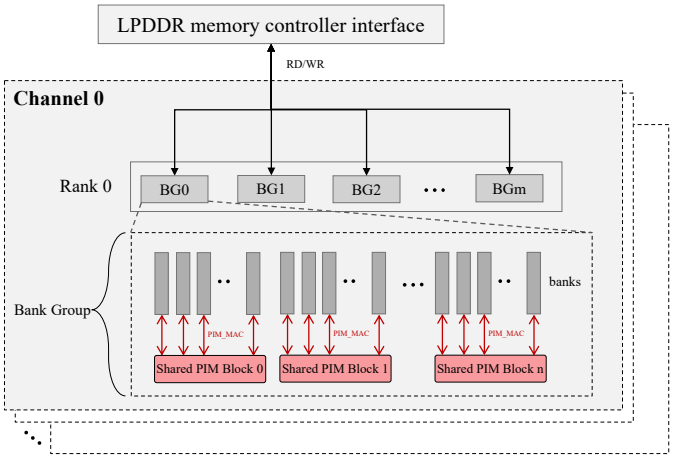

Fig. 1. LPDDR-PIM architecture based on Samsung's release [7]. A SIMD PIM block sits near the bank I/O and is driven by PIM commands. Samsung specifies two organizations: one PIM block per bank (b=1), or one block shared across a two-bank group (b=2).

PIM reduces the memory-wall penalty by co-locating arithmetic with data [11], [12]. DRAM-PIM spans 3D-stacked designs that place processing units in the logic die of memories such as HMC [13]–[15] and near-bank designs that integrate SIMD units beside the DRAM banks and drive them over the standard command bus [16]–[18]. LPDDR-PIM [19]–[22], which is dominant on mobile and edge SoCs where power and area are constrained, belongs to the second category. Samsung's LPDDR-PIM (illustrated in Fig. 1) is the primary public industrial reference: it places FP16/INT8 SIMD PIM blocks near the I/O boundary and drives them with dedicated DRAM and PIM commands so that host and PIM requests share the LPDDR channel [7], [17]. Samsung designs this architecture in two organizations: one PIM block per bank, or one block shared across a two-bank group. This architectural choice is central to LPDDR-PIM [7]. JEDEC's LPDDR6 roadmap now lists PIM as a standardization target [23], [24], signaling that LPDDR-PIM is becoming a mainstream edge-

*Corresponding author.

memory technology.

Pre-silicon evaluation of LPDDR-PIM requires a *DRAM simulator*: a model that replays memory-access traces as timing-constrained DRAM commands (`ACT`, `PRE`, `RD`, `WR`) on the command bus [25]–[28]. Modeling LPDDR-PIM adds two requirements: controller-visible PIM opcodes must appear in the command stream, and the simulator must enforce the bank contention they induce, since shared-block serialization are recoverable from an analytical model. We survey existing simulators against these criteria and select Ramulator 2.0 [28] as the extension base in §II. To the best of our knowledge, no open-source command-level LPDDR-PIM simulator exists.[1] We close this gap with *PIMScope*, the first open-source command-level LPDDR-PIM backend that simulates full transformer command traces,[2] and our contributions are summarized as follows:

1) **Extend** Ramulator 2.0 with PIM opcodes, a shared-PIM-block model, and a two-level timing model, plus a structural energy interface: JEDEC-grounded LPDDR timing is inherited from the base simulator, while PIM-specific timing and energy coefficients are independently parameterizable. Timing and energy coefficients are configured with literature-supported values in §III-B.
2) **Lower** a declarative transformer workload specification into backend-executable LPDDR-PIM command streams via a trace-lowering pipeline in §III-C.
3) **Evaluate** the LPDDR-PIM backend across 15 decoder-only models from five families, quantifying the shared-block design cost (1.17–1.80× slowdown) and the decode weight-preload overhead (18–38% in decode and. 2–3% in prefill), with direct weight-placement implications (§IV).

II. Background and Related Work

Evaluating an LPDDR-PIM design before silicon requires a *DRAM simulator*: a model that turns a memory-access trace into the DRAM commands (`ACT`, `PRE`, `RD`, `WR`) issued on the command bus and accounts for their latency under JEDEC timing and bank-state constraints. This *command-level* fidelity is what a PIM study needs: PIM exposes memory-side compute as additional commands that contend for the same bus and banks, so shared-block serialization cannot be recovered from an analytical model. Simulating LPDDR-PIM therefore places two requirements on top of a conventional DRAM simulator: controller-visible PIM opcodes in the command stream, and a resource model for the bank conflicts they induce. §II-A surveys existing simulators against these requirements. §II-B examines why no existing LPDDR-PIM backend has been publicly released.

A. Command-Level DRAM and PIM Simulators

A simulator that meets the two requirements above must first provide command-level DRAM timing, an open LPDDR5 specification, and an extensible controller interface. Table I maps existing tools onto these criteria. General-purpose simulators (DRAMSim3, DRAMSys, Ramulator 2.0) cover timing, but only DRAMSys and Ramulator 2.0 ship a LPDDR5 model [25], [26], [28]. PIM-oriented simulators target HMC, HBM2-PIM, or HBM3-PIM [5], [31]–[36], none of which model the LPDDR command interface. The gap is therefore an LPDDR5-capable, command-level simulator with native PIM support, and no existing tool occupies that row. Among LPDDR5-capable bases, Ramulator 2.0 is the better extension target: its configuration-driven command and controller interface was designed for new command types, whereas DRAMSys's SystemC/TLM stack is heavier to extend with controller-visible PIM opcodes.

TABLE I
Common DRAM Simulators Comparison

Simulator	Year	DRAM timing	LPDDR specification	Native PIM support
DRAMSim3 [25]	2020	✓	△ LPDDR3/4	✗
DRAMSys [26]	2020	✓	✓ LPDDR5	✗
Ramulator [27]	2016	✓	△ LPDDR3/4	✗
DRAMPower [30]	2012	✗	✓	✗
Ramulator-PIM [31]	2021	✓	△ [27]	✓HMC PIM
PIMSimulator [32]	2021	✓	✗	✓HBM2-PIM
PIMSys [33]	2024	✓	✗	✓HBM2-PIM
MultiPIM [34]	2021	✓	△ [27]	✓HMC PIM
MPU-Sim [35]	2022	✓	✗	✓3D-DRAM PIM
UPimulator [36]	2024	✓	✗	✓HBM2-PIM
AttAcc [5]	2024	✓	✓ [28]	✓HBM3-PIM
Ramulator 2.0 [28]	2024	✓	✓ LPDDR5	✗
Our work	2026	✓	✓ LPDDR5	✓LPDDR-PIM

✓: directly supported; △: partial, older-generation, or indirect support; ✗: not provided by the tool.

TABLE II
Evaluation backends used by recent LPDDR-PIM work

Work	Year	Evaluation backend	LPDDR specification	Public backend
CD-PIM [20]	2026	Modified Ramulator 2 [28]	✓	✗
FACIL [22]	2025	Modified NeuPIMs PIM simulator [4], built on DRAMsim3 [25]	△	✗
UMDAM [37]	2025	Extended Ramulator 2.0 [28] for LPDDR5-PIM; NPU modeled with AttAcc [5]	✓	✗
LP-Spec [19]	2025	PIMSimulator [32] + LLMCompass [38]	△	✗
Fold-PIM [21]	2025	Modified Ramulator 2 [28]	✓	✗
PIM-AI [39]	2024	UPMEM LLM Framework [40]	✗	△*
PIM-SHERPA [41]	2026	Real hardware	—	✗
LP5X-PIM Sim [29]	2026	Built on DRAMSim3 [25] and Ramulator [27]	✓	✗

✓: directly supported; △: partial, older-generation, or indirect support; ✗: not provided by the tool. ∗: UPMEM provides a public analytical PyTorch/YAML profiler, but does not release the PIM-AI hardware design.

B. The Missing Backend for LPDDR-PIM Architectures

Recent LPDDR-PIM work each rebuild a private evaluation stack (Table II). CD-PIM, UMDAM, and Fold-PIM independently extend Ramulator 2.0 [20], [21], [37]; FACIL modifies the NeuPIMs/DRAMSim3 stack [4], [22], [25]; LP-Spec pairs PIMSimulator with LLMCompass [19], [32], [38]; PIM-AI

[1] Concurrent with our work, Samsung released LP5X-PIM Sim [29], a cycle-accurate LPDDR5X-PIM simulator built on DRAMSim3 [25] and Ramulator [27], evaluated on GEMV kernels and not publicly released (§II-B).

[2] PIMScope is built on Ramulator 2.0 [28] and is available as open source at https://github.com/TtitiO/PIMScope.

uses an analytical YAML profiler [39], [40]. Concurrent with our work, Samsung's LP5X-PIM Sim [29] builds a cycle-accurate LPDDR5X-PIM simulator on DRAMSim3 [25] and Ramulator [27], modeling per-bank PIM blocks and evaluating GEMV kernels, but is not publicly released. That these groups independently chose Ramulator-based backends reflects that family's LPDDR5 timing model and extensible controller interface. PIMScope adopts the same modeling assumptions, namely PIM commands on the shared LPDDR bus and a two-bank shared-block organization, but is released as open infrastructure rather than tied to a single study.

The HBM-PIM community benefits from reusable simulation infrastructure (PIMSimulator, AttAcc, PIMSys [5], [32], [33]); LPDDR-PIM still lacks equivalent. AttAcc is the closest precedent, extending Ramulator 2.0 with controller-visible PIM commands [5]. PIMScope differs on three axes. First, the *target*: LPDDR-PIM uses command interface under mobile timing [7], whereas HBM3 targets stack organization [5]. Second, the *shared-block constraint*: PIMScope models a LPDDR-specific multi-bank exclusion rule and an SB/HAB state machine [7]. Third, the *cost model*: a two-level timing model separates issue interval from compute occupancy, paired with a two-layer energy decomposition.

III. Simulation Methodology and Backend Architecture

PIMScope is an extended simulation framework for evaluating autoregressive transformer inference on LPDDR-PIM architectures. The goal of PIMScope is not to reproduce a full software stack, but to expose the memory-system mechanisms that determine LPDDR-PIM performance. We build PIMScope by extending Ramulator 2.0 [28] at the command level rather than constructing a new simulator, as an LPDDR-PIM simulator inherits memory-related features from an LPDDR one. We first demonstrate the structure of Ramulator 2.0 as a DRAM simulator and identify what it lacks for native LPDDR-PIM support layer by layer; we then detail the extension that supplies it. Fig. 2 marks the result on the simulator stack: gray blocks are reused from Ramulator 2.0 unchanged, orange blocks are modified, and red blocks are new LPDDR-PIM components.

A. Ramulator 2.0 and LPDDR-PIM Support

Ramulator 2.0 is a command-level DRAM simulator that takes a configuration and a workload trace as input. It processes each request through a four-stage pipeline: 1) a *frontend* turns the trace into memory requests; 2) a *memory system* maps each request to the DRAM hierarchy and routes it to a channel; 3) a *DRAM device model* translates requests into timing-constrained DRAM commands and executes them; 4) an output stage emits statistics and logs. It fits LPDDR-PIM well: PIM commands travel the same command bus and obey the same bank and timing state as ordinary LPDDR commands, so a command-level DRAM simulator already models most of what an LPDDR-PIM simulator needs. The gaps are confined to the frontend and the device model, both of which assume all traffic is host reads and writes.

The frontend reads a trace and emits only host read and write requests, with no way to express a PIM operation. Supplying that PIM input is the role of the trace generator detailed in §III-C. The memory system maps a request address across channel, rank, bank group, bank, and row and dispatches it to the owning channel. PIM commands target the same hierarchy and need the same routing, so this layer is reused unchanged.

The device model is where the gaps concentrate, and they correspond one-to-one to the backend extension of §III-B. **G1 (opcodes):** Its command set covers only standard LPDDR commands (ACT, PRE, RD, WR) with no opcode for memory-side compute, and its controller, scheduling under timing and row-buffer state alone, cannot represent the PIM execution modes (single-bank entry, host all-bank setup, and PIM_MAC) under which Samsung's LPDDR-PIM (Fig. 1) gates command legality. **G2 (timing):** It models every command as a single latency, with no way to separate a MAC's issue interval from its compute occupancy. **G3 (shared block):** It tracks resources per bank, so it cannot enforce that one PIM block is shared between every two banks, and would let sibling banks issue concurrent MACs and overstate PIM parallelism. **G4 (energy):** It accounts only for JEDEC DRAM energy. The next subsection closes G1–G4 in turn, while the JEDEC LPDDR timing engine and DRAM organization are reused unchanged.

TABLE III
PIM timing and energy parameters introduced by PIMScope.

Symbol	Description	Basis
n_{RCD}	ACT→PIM_MAC wait	LPDDR5-6400 preset [28]
n_{II}	issue interval	8 cycles [19], [20]
L_{comp}	bank-occupancy ($L_{\mathrm{pipe}}+L_{\mathrm{move}}+L_{\mathrm{wb}}$)	8 cycles
b	banks per PIM block	1 (CD-PIM) [20], 2 (Samsung, LP-Spec) [7], [19]
E_{LPDDR}	base DRAM energy	DRAMPower IDD [30]
$e_{\mathrm{MAC}}^{\mathrm{INT8}}$	per-PIM_MAC (INT8)	0.35 pJ/MAC [20]
$e_{\mathrm{MAC}}^{\mathrm{FP16}}$	per-PIM_MAC (FP16)	0.69 pJ/MAC [17], [42]
e_{cell}	cell-to-PIM per 256 b	2.68 pJ [43], [44]
e_{VRF}	vector register-file access	3.17 pJ/256 b [45]
e_{SRF}	scalar register-file access	0.40 pJ/32 b [45]

B. Proposed LPDDR-PIM Extension

PIMScope closes the four gaps identified in §III-A with the four features described below. Table III summarizes the parameters they introduce.

G1: PIM commands and execution modes. To expose memory-side compute (the missing opcode of §III-A), PIMScope adds the LPDDR-PIM command set of Samsung's architecture [7]: single-bank (SB) and host all-bank (HAB) mode entry, activation broadcast (PIM_BCAST), and two MAC opcodes (per-bank PIM_MAC in SB mode and all-bank

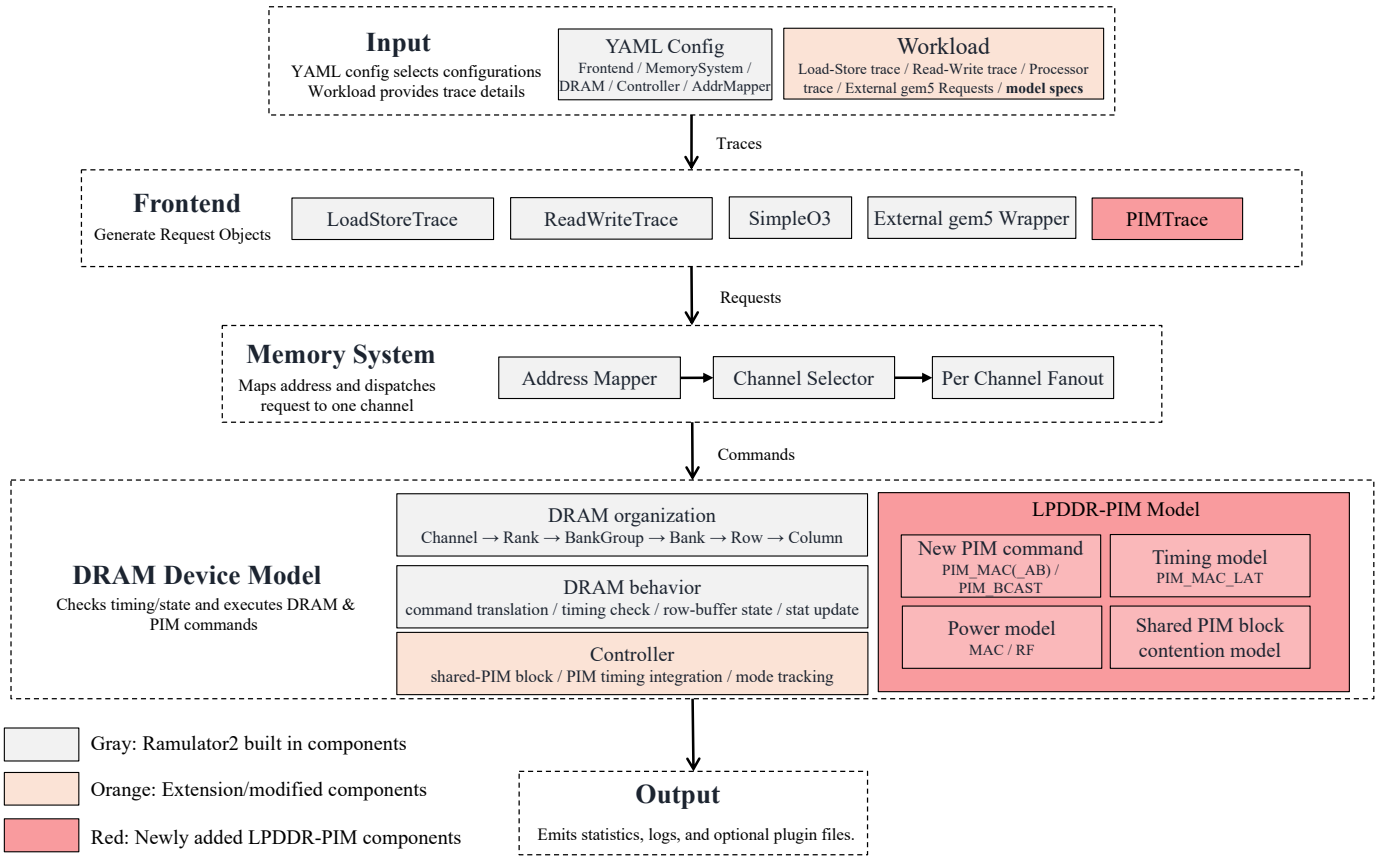

Fig. 2. PIMScope backend architecture as an extension of Ramulator 2.0. Gray blocks are reused from Ramulator 2.0, orange blocks are modified extension components, and red blocks are newly added LPDDR-PIM components.

broadcast `PIM_MAC_AB` in `HAB` mode). We model these as controller-visible commands on the existing DRAM command path rather than as a new level of the memory hierarchy: a separate PIM level would duplicate the address mapping and timing the DRAM organization already provides, whereas a command class lets host and PIM traffic share one scheduler and one timing engine.

G2: Two-level timing. PIM timing separates when a command may *issue* from how long it *occupies* the datapath. On the issue side, a `PIM_MAC` reuses the DRAM timing engine with two additional constraint edges: it waits n_{RCD} after a row activation, and successive MACs are spaced by an issue interval n_{II} that sets the command-bus throughput ceiling. On the completion side, once issued, a MAC holds its bank for $L_{\mathrm{comp}} = L_{\mathrm{pipe}} + L_{\mathrm{move}} + L_{\mathrm{wb}}$, the sum of the arithmetic pipeline depth, cell-to-PIM transfer, and writeback latencies (Table III). Modeling n_{II} and L_{comp} separately allows them to be swept independently, isolating interface throughput from compute speed; folding them into a single latency would couple the two axes and obscure whether a workload is issue-bound or compute-bound. Concretely, because issue is serialized by n_{II} while each MAC occupies its bank for L_{comp}, the number of banks a single command stream can keep simultaneously busy is bounded by $\lceil L_{\mathrm{comp}}/n_{\mathrm{II}} \rceil$, a command-bus limit no per-bank latency term alone would expose.

G3: Shared-PIM-block contention. A PIM block is shared by b banks, so PIMScope treats the *block* as the unit of compute exclusion: one block serves its b banks serially, one at a time. This single parameter spans both published LPDDR-PIM organizations (CD-PIM's dedicated per-bank compute units, b=1 [20], and Samsung/LP-Spec's block shared between two banks, b=2 [7], [19]; Fig. 1), so the same model evaluates both design points by changing b. The contention surfaces differently in the two MAC modes. In `SB` mode, a per-bank `PIM_MAC` issued to a bank whose block is already computing for its partner stalls until the block frees, even when the bank's own timing would otherwise permit issue. In `HAB` mode, an all-bank `PIM_MAC_AB` cannot drive its b banks simultaneously; the block walks them serially, so the broadcast MAC holds the datapath for $b\,L_{\mathrm{comp}}$ rather than L_{comp}. § III-C will show how the trace generator emits `SB` and `HAB` commands according to operand residency.

G4: Two-layer energy. PIMScope decomposes total energy as $E = E_{\mathrm{LPDDR}} + E_{\mathrm{PIM}}$, keeping the JEDEC-grounded base separate from PIM-specific cost. E_{LPDDR} follows the DRAMPower IDD method [30], while E_{PIM} sums per-command energy over the PIM command stream, where the `PIM_MAC` coefficient aggregates per-lane arithmetic, cell-to-PIM move-

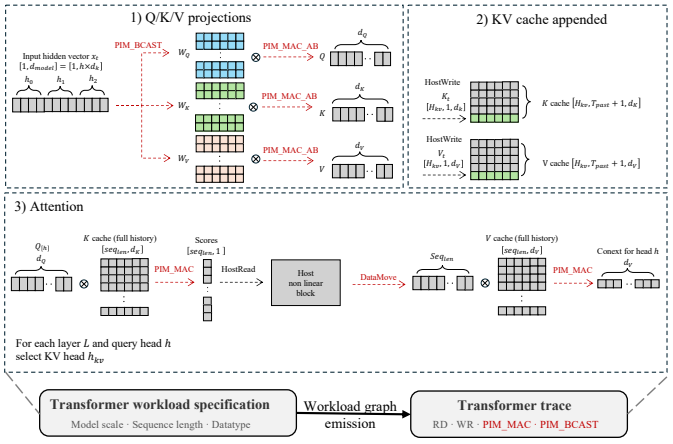

Fig. 3. Trace generation: a transformer specification is expanded into a semantic operator graph and then lowered into a backend-executable trace.

ment, interconnect, and register-file costs (Table III).

The coefficients are anchored in literatures. For INT8, CD-PIM reports 4.5 mW per compute unit at 32 MACs/cycle and 400 MHz on a TSMC 28 nm LPDDR5-based design [20], yielding $4.5\,\text{mW}/(400\,\text{MHz} \times 32) = 0.35\,\text{pJ/MAC}$. For FP16, P3-LLM reports 0.69 pJ/MAC on an HBM-PIM design [42], consistent with the Samsung INT8/FP16 ratio [17]. Cell-to-PIM movement is 2.68 pJ per 256-bit block from O'Connor's FGDRAM model [43], register-file costs (e_{VRF}, e_{SRF}) follow the partial-sum RF measurements of Gudaparthi et al. [45].

C. Trace Generation and Opcode Lowering

An offline trace compiler closes the frontend gap by translating a declarative transformer specification (layer count, hidden dimension, attention and KV-head counts, FFN variant, and optional MoE) into a backend-executable command stream in two stages (Fig. 3): a semantic stage emits a operator graph, and a lowering stage maps each kind to concrete LPDDR-PIM opcodes. The figure illustrates one decode step: Q/K/V projection broadcasts the input via `PIM_BCAST` and emits all-bank `PIM_MAC_AB`; the new K/V vectors append to the KV cache as host writes; attention multiplies the query against the K cache (`PIM_MAC`), the host computes softmax, and the score multiplies the V cache (`PIM_MAC`).

Opcode dispatch is driven by operand residency. *Weight-stationary* kinds (Q/K/V/O projections, FFN, and MoE expert and router layers), in which weights are preloaded per bank and a single activation vector is broadcast to all banks, lower to `PIM_MAC_AB` in `HAB` mode. *Data-stationary* kinds (attention score and context), in which each bank holds a distinct KV-cache tile, lower to per-bank `PIM_MAC` in `SB` mode. This split reflects the operand's physical placement (broadcast-shared weights versus bank-private KV-cache tiles), a distinction both LPDDR-PIM designs make; it is orthogonal to the b parameter of §III-B, which is what separates the two designs and applies to *both* paths.

Because fine-grained bank interleaving is what exposes inter-bank parallelism, the attention path would, if materialized as one record per (bank, tile), expand a single large model to of trace. PIMScope instead emits one *compact* `PIM_MAC` record carrying a bank-rotation descriptor (bank sequence, interleave depth, and dependency-column stride); the frontend expands it into the interleaved per-bank issue stream at issue time. This keeps traces tractable for workloads while reproducing the exact issue order, and lets independent banks in different PIM blocks execute concurrently while same-block banks serialize per §III-B.

Data movement follows a residency predicate: weights already resident in a bank emit no traffic (steady-state), a cold-start load emits `WRITE`, and an activation broadcast emits `PIM_BCAST` in `HAB` mode. Operators without a backend opcode, namely softmax, elementwise nonlinearities, MoE top-k, and dispatch/combine, are retained as ordering fences but emit no commands.

IV. Evaluations

We evaluate PIMScope through two experiments, each of which first establishes that the backend faithfully enforces a modeled constraint, then applies that constraint to expose workload-dependent behavior on real decoder-only transformer command streams.

1) *Shared-block design cost comparsion* (§IV-B): executing full transformer traces under per-bank ($b = 1$) and two-bank shared ($b = 2$) PIM blocks shows the modeled all-bank latency scaling resolve to a workload-dependent 1.17–1.80× end-to-end slowdown that tracks command-stream composition.
2) *Weight-residency cost comparison* (§IV-C): the complete pipeline runs across 15 autoregressive workloads from five families and surfaces a decode/prefill cold-start asymmetry with a direct weight-placement implication.

A. Experimental Setup

Table IV gives the DRAM and decode/prefill configurations; PIM timing and energy parameters are those of §III (Table III). We sweep the per-block sharing factor $b \in \{1, 2\}$ in §IV-B and fix it at the Samsung value b=2 elsewhere. The workload set is 15 decoder-only models from five families (Llama2, OPT, Qwen2.5, Gemma, Mixtral), each issued as a bounded decode or prefill command stream by the trace compiler of §III-C. We run every workload under two weight-residency regimes, which the data-movement predicate of §III-C resolves into different command streams. In steady-state, weights are assumed pre-resident in their home banks, so the stream carries only PIM compute and KV-cache traffic; this isolates the cost of PIM execution itself. In cold-start, those weights are first materialized by explicit `WRITE` traffic before any compute, modeling the one-time preload a freshly scheduled model incurs. The difference between the two regimes is therefore exactly the weight-preload cost, which §IV-C quantifies.

B. End-to-End Cost of PIM Block Sharing

The per-block sharing factor b of §III-B is a concrete design fork in the LPDDR-PIM literature: CD-PIM gives each bank

TABLE IV
DRAM AND WORKLOAD CONFIGURATIONS. PIM TIMING AND ENERGY PARAMETERS ARE LISTED IN TABLE III.

Parameter	**Value**
Memory standard	LPDDR5-6400
Clock period	$t_{\mathrm{CK}} = 1.25\,\mathrm{ns}$ (CK, CKR 4:1)
Organization	1 ch, 1 rank, 4 BG $\times$ 4 banks
Scheduling	FR-FCFS
Datatype	INT8 (256-bit SIMD, 32 lanes)
Decode configuration	$\mathrm{past\_len} = 1024,\ \mathrm{seq\_len} = 1$
Prefill configuration	$\mathrm{prompt\_len} = 12$

TABLE V
PER-BANK (b=1) VS. TWO-BANK SHARED (b=2) (STEADY-STATE).

Workload	**MAC frac.**	b=1 **(MCycles)**	b=2 **(MCycles)**	**Slowdown** $(b{=}2)/(b{=}1)$
OPT-125M	0.72	9.1	10.7	1.17×
OPT-350M	0.73	25.8	31.4	1.22×
OPT-1.3B	0.78	63.4	85.1	1.34×
Gemma-2-9B	0.83	301.0	448.5	1.49×
Llama2-7B	0.83	231.8	346.8	1.50×
Gemma-2B	0.84	68.4	103.5	1.51×
Qwen2.5-14B	0.84	455.5	689.8	1.51×
Llama2-13B	0.85	412.4	637.1	1.54×
Qwen2.5-7B	0.86	208.1	323.6	1.56×
Gemma-7B	0.86	243.7	380.7	1.56×
Qwen2.5-32B	0.89	872.8	1424.0	1.63×
Llama2-70B	0.89	1863.6	3072.2	1.65×
Qwen2.5-72B	0.90	1898.0	3137.6	1.65×
Gemma-2-27B	0.95	1118.0	1989.5	1.78×
Mixtral-8x7B	0.96	1102.9	1990.0	1.80×

a dedicated compute unit (b=1), whereas Samsung's design and LP-Spec share one PIM block between every two banks (b=2) [7], [19], [20]. Sharing constrains the all-bank path: a shared block serves its b banks one at a time, so the model scales the all-bank MAC latency by b. We measure how that microarchitectural cost propagates to the application level by executing all 15 decode traces under both factors with every other parameter fixed (Table V).

The realized slowdown spans 1.17–1.80× and rises monotonically with the PIM-MAC fraction, which is dominated by the weight-stationary projection and FFN work routed to the all-bank path by the residency split of §III-C. Host KV-cache `READ` traffic, invariant to b, offsets the scaled all-bank latency, so a projection-light workload such as OPT-125M (MAC fraction 0.72) slows by only 1.17×. As the FFN share grows, the all-bank latency scaling governs a larger fraction of the stream and the slowdown climbs toward the b=2 bound. Mixtral-8x7B is the limiting case: its routed experts make the stream almost purely weight-stationary, so its slowdown lands nearest the bound at 1.80× (MAC fraction 0.96).

C. Weight-Residency Cost Comparison across Models

With the resource model and front-end command count established, we run the complete pipeline (specification, semantic record emission, opcode lowering, backend execution) across all 58 traces: 30 decode (15×2) and 28 prefill (14×2) in both steady-state and cold-start modes (Fig. 4). Prefill excludes Mixtral-8x7B because the generator implements the MoE expert path only for decode, a frontend scope limitation rather than a backend constraint. Model dimensions span $d_{\mathrm{model}} = 768$ (OPT-125M) to 8192 (Llama2-70B, Qwen2.5-72B), producing PIM MAC command counts from 3.2×10^6 to 2.6×10^{10}. Every trace reports an exact match between issued and completed opcode counts, establishing *pipeline determinism* as a floor; the stronger timing-constraint evidence comes from §IV-B, since count-matching alone would also hold for a simulator that enforced no constraint.

a) Cold-start asymmetry exposes a weight-placement target.: The characterization's primary finding is a sharp decode/prefill asymmetry that bears directly on memory placement: comparing steady-state (weights pre-resident) against cold-start (explicit `WRITE` materialization), cold-start adds 18–38% cycles in decode but only 2–3% in prefill. The cause is the ratio of one-time weight traffic to per-step compute. Materializing the weights moves a volume set by the parameter count, a space complexity of $\Theta(d_{\mathrm{model}}^2 \cdot n_{\mathrm{layers}})$, so the one-time load is a non-vanishing fraction of the decode step's cycles: 18% for OPT-125M ($d_{\mathrm{ffn}} = 3072$) rising to 38% for Gemma-2-27B ($d_{\mathrm{ffn}} = 73{,}728$), with Mixtral-8x7B at 35% because all 8 expert weight sets are preloaded while only the top-2 contribute MACs per step. A prefill step processes a prompt of length L and spreads the same load by a factor of L, cutting the overhead to $\leq 3\%$. Weight-placement optimization (bank partitioning, partial preload, migration) therefore yields its largest gains in decode.

V. DISCUSSIONS

The measured 18–38% decode penalty suggests two potential optimizations. At the controller level, persistent weight residency can keep reusable weights in bank-local regions, while double buffering can overlap the current computation with loading the next weight set when resources permit. At the algorithm level, a lightweight predictor can anticipate the weight likely to be needed in the next layer and prefetch it before it is requested.

Like every pre-silicon PIM study, PIMScope's timing and energy parameters draw from literature figures (Table III); we therefore make a bounded claim. The backend is validated for *structural and command-level correctness* (exact issued-versus-completed command matching across all 58 traces spanning 15 models), and the sharing experiment (§IV-B) confirms that the timing model enforces the shared-PIM-block constraint, with measured slowdowns tracking the PIM-MAC fraction monotonically from 1.17× to 1.80×. We claim no absolute timing or energy accuracy against real LPDDR-PIM architectures: all parameters are configurable, so external calibration is straightforward as public LPDDR-PIM device data becomes available. PIMScope models only the memory-side command stream; host-side operations (softmax, nonlinearities, MoE top-k/dispatch/combine) remain out of scope. For full-system analysis, PIMScope exposes an external

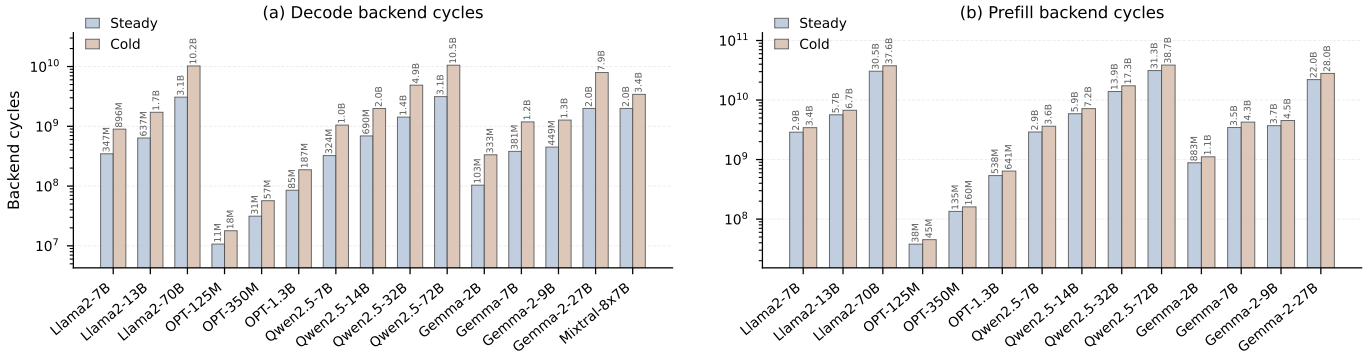

Fig. 4. Full backend simulation under $b=2$ across 15 decode (left) and 14 prefill (right) configurations. Bars show total cycles per step in cold-start (weight preload) and steady-state (weights pre-resident) modes.

request path for memory accesses. An adapter can translate CPU/NPU memory requests or traces into PIMScope requests. Within these boundaries, PIMScope provides the first open-source command-level simulation infrastructure for LPDDR-PIM architectural analysis, enabling quantitative what-if studies that previously required either private backends or analytical approximations.

VI. Conclusions

On-device transformer inference is bottlenecked by autoregressive-decode memory bandwidth, making LPDDR-PIM an attractive option for placing compute near the DRAM banks. Yet open command-level infrastructure remains scarce: prior proposals rely on private backends, and the concurrent LP5X-PIM Sim [29] is unreleased and evaluates GEMV kernels rather than full models. Questions such as the cost of sharing one PIM block across banks could therefore be argued only with isolated kernels or analytical models, not measured on real transformer command streams. This work closes that gap with PIMScope, an open-source (https://github.com/TtitiO/PIMScope) command-level LPDDR-PIM backend built on Ramulator 2.0 that adds native PIM opcodes, a shared-PIM-block model, and a two-level timing model with a structural energy interface, plus a trace-to-opcode lowering path from declarative transformer specifications to backend-executable command streams. We tested it on 15 decoder-only architectures from five families through full emitted-command execution, confirming that it enforces the shared-PIM-block constraint and runs all workloads deterministically. Future work should add mixed-phase traces, richer datatype-aware PIM resources, head overlap scheduling, additional model families, and external calibration as public LPDDR-PIM device data becomes available.

Acknowledgment

This work was supported in part by the National Science Foundation of China under Grant No. 62201340, in part by the Shanghai Sci-tech Co-research Program No. 25HB2704100. The authors thank the developers of Ramulator tools [27], [28], [31], whose open-source simulation infrastructure made this work possible.

References

[1] W. A. Wulf and S. A. McKee, "Hitting the memory wall: implications of the obvious," *SIGARCH Comput. Archit. News*, vol. 23, no. 1, p. 20–24, Mar. 1995. [Online]. Available: https://doi.org/10.1145/216585.216588

[2] A. Boroumand, S. Ghose, Y. Kim, R. Ausavarungnirun, E. Shiu, R. Thakur, D. Kim, A. Kuusela, A. Knies, P. Ranganathan, and O. Mutlu, "Google workloads for consumer devices: Mitigating data movement bottlenecks," *SIGPLAN Not.*, vol. 53, no. 2, p. 316–331, Mar. 2018. [Online]. Available: https://doi.org/10.1145/3296957.3173177

[3] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, J. Heek, K. Xiao, S. Agrawal, and J. Dean, "Efficiently scaling transformer inference," in *Proceedings of Machine Learning and Systems*, D. Song, M. Carbin, and T. Chen, Eds., vol. 5. Curan, 2023, pp. 606–624. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2023/file/c4be71ab8d24cdfb45e3d06dbfca2780-Paper-mlsys2023.pdf

[4] G. Heo, S. Lee, J. Cho, H. Choi, S. Lee, H. Ham, G. Kim, D. Mahajan, and J. Park, "Neupims: Npu-pim heterogeneous acceleration for batched llm inferencing," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ser. ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 722–737. [Online]. Available: https://doi.org/10.1145/3620666.3651380

[5] J. Park, J. Choi, K. Kyung, M. J. Kim, Y. Kwon, N. S. Kim, and J. H. Ahn, "Attacc! unleashing the power of pim for batched transformer-based generative model inference," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 103–119. [Online]. Available: https://doi.org/10.1145/3620665.3640422

[6] S. Williams, A. Waterman, and D. Patterson, "Roofline: an insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, p. 65–76, Apr. 2009. [Online]. Available: https://doi.org/10.1145/1498765.1498785

[7] B. Kim, S. Cha, S. Park, J. Lee, S. Lee, S.-h. Kang, J. So, K. Kim, J. Jung, J.-G. Lee, S. Lee, Y. Paik, H. Kim, J.-S. Kim, W.-J. Lee, Y. Ro, Y. Cho, J. H. Kim, J. Song, J. Yu, S. Lee, J. Cho, and K. Sohn, "The breakthrough memory solutions for improved performance on llm inference," *IEEE Micro*, vol. 44, no. 3, pp. 40–48, 2024.

[8] JEDEC Solid State Technology Association, "Low Power Double Data Rate (LPDDR) 5/5X," JEDEC Standard JESD209-5C, Jul. 2023, accessed: 2026-05-25. [Online]. Available: https://www.jedec.org/standards-documents/docs/jesd209-5c

[9] ——, "JEDEC publishes new and updated standards for low power memory devices used in 5G and AI applications," https://www.jedec.org/news/pressreleases/jedec-publishes-new-and-updated-standards-low-power-memory-devices-used-5g-and-ai, Jul. 2021, accessed: 2026-06-06.

[10] L. Steiner, M. Jung, M. Huonker, and N. Wehn, "Unveiling the real performance of lpddr5 memories," in *Proceedings of the 2022 International Symposium on Memory Systems*, ser. MEMSYS '22. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: https://doi.org/10.1145/3565053.3565062

[11] O. Mutlu, S. Ghose, J. Gómez-Luna, and R. Ausavarungnirun, *A Modern Primer on Processing in Memory*. Singapore: Springer Nature Singapore, 2023, pp. 171–243. [Online]. Available: https://doi.org/10.1007/978-981-16-7487-7_7

[12] S. Ghose, A. Boroumand, J. S. Kim, J. Gómez-Luna, and O. Mutlu, "Processing-in-memory: A workload-driven perspective," *IBM Journal of Research and Development*, vol. 63, no. 6, pp. 3–1, 2019.

[13] Micron Technology, Inc., *Hybrid Memory Cube – HMC Gen2: MT43A4G40200 – 2GB 4H DRAM Stack*, Micron Technology, Inc., Feb. 2018, rev. H, Datasheet. [Online]. Available: https://www.mouser.com/datasheet/2/671/hmc_gen2-1382047.pdf

[14] J. Ahn, S. Hong, S. Yoo, O. Mutlu, and K. Choi, "A scalable processing-in-memory accelerator for parallel graph processing," in *Proceedings of the 42nd Annual International Symposium on Computer Architecture (ISCA)*, 2015, pp. 105–117.

[15] ——, "PIM-enabled instructions: A low-overhead, locality-aware processing-in-memory architecture," in *Proceedings of the 42nd Annual International Symposium on Computer Architecture (ISCA)*, 2015, pp. 336–348.

[16] J. H. Kim, S.-h. Kang, S. Lee, H. Kim, W. Song, Y. Ro, S. Lee, D. Wang, H. Shin, B. Phuah, J. Choi, J. So, Y. Cho, J. Song, J. Choi, J. Cho, K. Sohn, Y. Sohn, K. Park, and N. S. Kim, "Aquabolt-xl: Samsung hbm2-pim with in-memory processing for ml accelerators and beyond," in *2021 IEEE Hot Chips 33 Symposium (HCS)*, 2021, pp. 1–26.

[17] J. H. Kim, S.-H. Kang, S. Lee, H. Kim, Y. Ro, S. Lee, D. Wang, J. Choi, J. So, Y. Cho, J. Song, J. Cho, K. Sohn, and N. S. Kim, " Aquabolt-XL HBM2-PIM, LPDDR5-PIM With In-Memory Processing, and AXDIMM With Acceleration Buffer ," *IEEE Micro*, vol. 42, no. 03, pp. 20–30, May 2022. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/MM.2022.3164651

[18] H. Kim, H. Park, T. Kim, K. Cho, E. Lee, S. Ryu, H.-J. Lee, K. Choi, and J. Lee, "Gradpim: A practical processing-in-dram architecture for gradient descent," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 249–262.

[19] S. He, Z. Zhu, Y. He, and T. Jia, "Lp-spec: Leveraging lpddr pim for efficient llm mobile speculative inference with architecture-dataflow co-optimization," in *2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, 2025, pp. 1–9.

[20] Y. Lin, C. Fang, X. Song, Q. Wu, A. Jiang, Y. Bai, and L. Du, "Cd-pim: A high-bandwidth and compute-efficient lpddr5-based pim for low-batch llm acceleration on edge-device," 2026. [Online]. Available: https://arxiv.org/abs/2601.12298

[21] K. Jeun, H. Kim, and E. Lee, "Fold-pim: A cost-efficient lpddr5-based pim for on-device slms," *IEEE Computer Architecture Letters*, vol. 24, no. 1, pp. 185–188, 2025.

[22] S. H. Seo, J. Kim, D. Lee, S. Yoo, S. Moon, Y. Park, and J. W. Lee, "Facil: Flexible dram address mapping for soc-pim cooperative on-device llm inference," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2025, pp. 1720–1733.

[23] JEDEC Solid State Technology Association, "JEDEC® Previews LPDDR6 Roadmap Expanding LPDDR into Data Centers and Processing-in-Memory," https://www.jedec.org/news/pressreleases/jedec%C2%AE-previews-lpddr6-roadmap-expanding-lpddr-data-centers-and-processing-memory, Apr. 2026, accessed: 2026-05-25.

[24] TrendForce, "[News] Samsung and SK hynix Reportedly Unite to Standardize LPDDR6-PIM for On-device AI," https://www.trendforce.com/news/2024/12/02/news-samsung-and-sk-hynix-reportedly-unite-to-standardize-lpddr6-pim-for-on-device-ai/, Dec. 2024, accessed: 2026-05-25.

[25] S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. Jacob, "Dramsim3: A cycle-accurate, thermal-capable dram simulator," *IEEE Computer Architecture Letters*, vol. 19, no. 2, pp. 106–109, 2020.

[26] L. Steiner, M. Jung, F. S. Prado, K. Bykov, and N. Wehn, "Dramsys4.0: A fast and cycle-accurate systemc/tlm-based dram simulator," in *Embedded Computer Systems: Architectures, Modeling, and Simulation*, A. Orailoglu, M. Jung, and M. Reichenbach, Eds. Cham: Springer International Publishing, 2020, pp. 110–126.

[27] Y. Kim, W. Yang, and O. Mutlu, "Ramulator: A fast and extensible dram simulator," *IEEE Computer Architecture Letters*, vol. 15, no. 1, pp. 45–49, 2016.

[28] H. Luo, Y. C. Tuğrul, F. N. Bostancı, A. Olgun, A. G. Yağlıkçı, and O. Mutlu, "Ramulator 2.0: A modern, modular, and extensible dram simulator," *IEEE Computer Architecture Letters*, vol. 23, no. 1, pp. 112–116, 2024.

[29] S. Cha, J. Choi, B. Kim, Y. Paik, S. Lee, and K. Sohn, "Lp5x-pim sim: A high-fidelity hw/sw integrated simulator for lpddr5x-pim," 2026. [Online]. Available: https://arxiv.org/abs/2606.00636

[30] K. Chandrasekar, C. Weis, Y. Li, S. Goossens, M. Jung, O. Naji, B. Akesson, N. Wehn, and K. Goossens, "DRAMPower: Open-source dram power and energy estimation tool," https://github.com/tukl-msd/DRAMPower, 2012, gitHub repository, accessed: 2026-05-25.

[31] CMU SAFARI Research Group, "Ramulator-PIM: ZSim+Ramulator – a Processing-in-Memory simulation framework," https://github.com/CMU-SAFARI/ramulator-pim, 2021, gitHub repository. Accessed: 2026-05-25.

[32] SAITPublic, "PIMSimulator: Processing-In-Memory Simulator," https://github.com/SAITPublic/PIMSimulator, 2021, gitHub repository. Accessed: 2026-05-25.

[33] D. Christ, L. Steiner, M. Jung, and N. Wehn, "Pimsys: A virtual prototype for processing in memory," in *Proceedings of the International Symposium on Memory Systems*, ser. MEMSYS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 26–33. [Online]. Available: https://doi.org/10.1145/3695794.3695797

[34] C. Yu, S. Liu, and S. Khan, "Multipim: A detailed and configurable multi-stack processing-in-memory simulator," *IEEE Computer Architecture Letters*, vol. 20, no. 1, pp. 54–57, 2021.

[35] X. Xie, P. Gu, J. Huang, Y. Ding, and Y. Xie, "Mpu-sim: A simulator for in-dram near-bank processing architectures," *IEEE Computer Architecture Letters*, vol. 21, no. 1, pp. 1–4, 2022.

[36] B. Hyun, T. Kim, D. Lee, and M. Rhu, "Pathfinding future pim architectures by demystifying a commercial pim technology," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2024, pp. 263–279.

[37] H. Huang, "Umdam: A unified data layout and dram address mapping for heterogenous npu-pim," 2025. [Online]. Available: https://arxiv.org/abs/2511.03293

[38] H. Zhang, A. Ning, R. B. Prabhakar, and D. Wentzlaff, "Llmcompass: Enabling efficient hardware design for large language model inference," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024, pp. 1080–1096.

[39] C. Ortega, Y. Falevoz, and R. Ayrignac, "Pim-ai: A novel architecture for high-efficiency llm inference," 2024. [Online]. Available: https://arxiv.org/abs/2411.17309

[40] "UPMEM LLM Framework," https://github.com/upmem/upmem_llm_framework, 2024, mIT-licensed GitHub repository. Accessed: 2026-05-27.

[41] S. Lee, S. Cha, H. Kim, S. Seo, Y. Ro, S. Lee, B. Kim, Y. Park, K. Sohn, S. Lee, and J. Yu, "Pim-sherpa: Software method for on-device llm inference by resolving pim memory attribute and layout inconsistencies," 2026. [Online]. Available: https://arxiv.org/abs/2603.09216

[42] Y. Chen, C. Fang, X. Dai, Y. Wu, T. Tambe, M. Verhelst, and M. S. Abdelfattah, "P3-LLM: An integrated NPU-PIM accelerator for edge LLM inference using hybrid numerical formats," 2026, accepted to ISCA 2026. [Online]. Available: https://arxiv.org/abs/2511.06838

[43] M. O'Connor, N. Chatterjee, D. Lee, J. Wilson, A. Agrawal, S. W. Keckler, and W. J. Dally, "Fine-grained dram: Energy-efficient dram for extreme bandwidth systems," in *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*, 2017, pp. 41–54.

[44] B. Khabbazan, M. Riera, and A. González, "Towards efficient lut-based pim: A scalable and low-power approach for modern workloads," *IEEE Transactions on Parallel and Distributed Systems*, 2026.

[45] S. Gudaparthi, S. Narayanan, R. Balasubramonian, E. Giacomin, H. Kambalasubramanyam, and P.-E. Gaillardon, "Wire-aware architecture and dataflow for cnn accelerators," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO-52. New York, NY, USA: Association for Computing Machinery, 2019, p. 1–13. [Online]. Available: https://doi.org/10.1145/3352460.3358316